ShoreGuard: Knowledge Transfer Document

Design to Implementation: Lessons Learned Group X | ELEC1005 | May 2026

Overview ShoreGuard is a Power Apps canvas app connecting beach volunteers with facility coordinators, backed by three Microsoft SharePoint Lists. This document captures what we learned moving from design to implementation so future developers can pick up where we left off.

What Changed from Design to Implementation

We assumed a generic database backend — everything runs through SharePoint Lists using Power Apps functions (Patch(), Filter(), LookUp()) instead of SQL. Authentication had no native Power Apps solution against custom lists, so we used conditional logic comparing stored email/password fields — functional for a prototype but not production-secure. Several features were descoped due to time: SharePoint Patch() on sign-up forms, full dynamic roster filtering, and alert response logic are partially or not implemented, though the UI is in place for all of them.

Key Technical Lessons

- **Patch()** handles both creating and updating SharePoint items — use it for all form submissions.
- **Filter() and LookUp()** are your query tools — LookUp() for single records (login), Filter() for rosters.
- **Set()** is the only way to pass data between screens — store the logged-in user globally at login with Set(gCurrentUser, Lookup(...)) so every screen can access it without re-querying.
- Keep SharePoint field names simple with no spaces — Power Apps struggles to reference them otherwise.
- Test SharePoint connector permissions early — delegation errors only appear at runtime.
- Use Navigate() with role conditions for routing — trigger it from the login button based on account type.

What We'd Do Differently

Connect SharePoint before building screens (not after), implement Patch() calls screen by screen rather than as a late phase, and use Power Apps Monitor from day one to catch slow or failing SharePoint calls early.

Next Steps for Future Developers: Complete Patch() integration on sign-up forms → build alert accept/decline logic → replace plain-text passwords with Azure AD authentication → add push notifications for emergency alerts.